Downloading and Extracting data

```
!wget -q --show-progress https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz
!gunzip en.openfoodfacts.org.products.csv.gz
```

```
en.openfoodfacts.or 100%[===================>]   1.18G  34.5MB/s    in 40s
```

Loading data

```
import pandas as pd

cols = ['product_name', 'categories_tags', 'sugars_100g',
        'proteins_100g', 'fat_100g', 'fiber_100g', 'ingredients_text']

df = pd.read_csv('en.openfoodfacts.org.products.csv',
                 sep='\t',
                 usecols=cols,
                 nrows=500_000,
                 low_memory=False)

print(df.shape)
df.head()
```

```
(500000, 7)
```

	product_name	categories_tags	ingredients_text	fat_100g	sugars_100g	fiber_100g	proteins_100g
0	Limonade artisanale a la rose	NaN	NaN	NaN	NaN	NaN	NaN
1	M&M white	NaN	Weizenmehl, Rapsöl, Speisesalz, 1,7% Meersalz,...	NaN	NaN	NaN	NaN
2	Chocolate n3	NaN	NaN	NaN	NaN	NaN	NaN
3	Pâte de fruits	NaN	NaN	NaN	NaN	NaN	NaN
4	Paleta gran reserva - Sierra nevada-	en:beverages-and-beverages-preparations,en:bev...	Thiamin, Biotin, Chromium, Garcinia cambogia f...	NaN	NaN	NaN	NaN

Story 1: Clean Up

```
# 1. Drop rows missing the key nutrition columns or product name
df_clean = df.dropna(subset=['product_name', 'sugars_100g', 'proteins_100g'])

print(f"After dropping nulls: {df_clean.shape}")

# 2. Filter out biologically impossible values
# Nothing can exceed 100g per 100g of food, nothing can be negative
df_clean = df_clean[
    (df_clean['sugars_100g'] >= 0)    & (df_clean['sugars_100g'] <= 100) &
    (df_clean['proteins_100g'] >= 0)  & (df_clean['proteins_100g'] <= 100) &
    (df_clean['fat_100g'] >= 0)       & (df_clean['fat_100g'] <= 100)
]

print(f"After removing outliers: {df_clean.shape}")

# 3. Drop duplicates
df_clean = df_clean.drop_duplicates(subset=['product_name'])

print(f"After deduplication: {df_clean.shape}")
df_clean.head()
```

```
After dropping nulls: (105154, 7)
After removing outliers: (104760, 7)
After deduplication: (82207, 7)
```

	product_name	categories_tags	ingredients_text	fat_100g	sugars_100g	fiber_100g	proteins_100g
693	Pinto Bean	en:asian-style-ready-meal	NaN	10.20000	4.900000	3.800000	17.500000
694	Croquetas de bacalao	NaN	NaN	12.10000	1.900000	NaN	5.900000
695	Keto & GF Granola	NaN	NaN	54.83871	3.225806	9.677419	19.677419
801	Ben's Pure Maple Cream	NaN	NaN	30.28000	37.720000	NaN	5.680000
833	Guimauve chocolat smarties	NaN	NaN	19.00000	65.000000	NaN	5.700000

Handling NaN values in the critical columns

```
# fiber_100g NaN → fill with 0 (absence of fiber data ≠ no fiber, but it's the safest assumption)
df_clean['fiber_100g'] = df_clean['fiber_100g'].fillna(0)

# ingredients_text NaN -> only needed for Bonus Story 5
# Leaving it as NaN for now, I'll filter it out when we get there

# Confirm no NaNs remain in critical columns
print(df_clean[['sugars_100g', 'proteins_100g', 'fat_100g', 'fiber_100g']].isnull().sum())
print(f"\ningredients_text NaNs: {df_clean['ingredients_text'].isnull().sum()}")
print(f"categories_tags NaNs: {df_clean['categories_tags'].isnull().sum()}")
```

```
sugars_100g      0
proteins_100g    0
fat_100g         0
fiber_100g       0
dtype: int64

ingredients_text NaNs: 47201
categories_tags NaNs: 43950
```

Story 2: Category Wrangler

```
def assign_category(tags):
    if pd.isna(tags):
        return 'Other'
    tags = tags.lower()
    if any(k in tags for k in ['chocolate', 'candy', 'confectionery', 'gummy', 'sweet-snack', 'sugar']):
        return 'Candy & Chocolate'
    if any(k in tags for k in ['chip', 'crisp', 'pretzel', 'popcorn', 'cracker']):
        return 'Chips & Crisps'
    if any(k in tags for k in ['cookie', 'biscuit', 'wafer', 'brownie']):
        return 'Cookies & Biscuits'
    if any(k in tags for k in ['granola', 'cereal-bar', 'energy-bar', 'protein-bar', 'snack-bar']):
        return 'Bars & Granola'
    if any(k in tags for k in ['nut', 'seed', 'almond', 'peanut', 'cashew', 'pistachio']):
        return 'Nuts & Seeds'
    if any(k in tags for k in ['yogurt', 'yoghurt', 'cheese', 'dairy']):
        return 'Dairy Snacks'
    if any(k in tags for k in ['jerky', 'meat-snack', 'dried-meat']):
        return 'Meat Snacks'
    if any(k in tags for k in ['fruit', 'dried-fruit', 'raisin']):
        return 'Fruit Snacks'
    return 'Other'

df_clean['primary_category'] = df_clean['categories_tags'].apply(assign_category)

# Check distribution
print(df_clean['primary_category'].value_counts())
```

```
primary_category
Other                65916
Candy & Chocolate     5202
Dairy Snacks          4691
Fruit Snacks          2965
Nuts & Seeds          1826
Chips & Crisps        1459
Meat Snacks             61
Bars & Granola          53
```

```
Cookies & Biscuits       34
Name: count, dtype: int64
```

The above category bucketing produces results but the "Other" category is too dominant. This is because most products have NaN or unrecognized tags.

Trying to make matching broader to fix that

```
def assign_category(tags):
    if pd.isna(tags):
        return 'Other'
    tags = tags.lower()
    if any(k in tags for k in ['chocolate', 'candy', 'confectionery', 'gummy', 'sweet-snack',
                               'sugar', 'caramel', 'fudge', 'marshmallow']):
        return 'Candy & Chocolate'
    if any(k in tags for k in ['chip', 'crisp', 'pretzel', 'popcorn', 'cracker', 'corn-snack', 'puffed', 'tortilla']):
        return 'Chips & Crisps'
    if any(k in tags for k in ['cookie', 'biscuit', 'wafer', 'brownie', 'cake', 'pastry', 'muffin', 'baked']):
        return 'Cookies & Biscuits'
    if any(k in tags for k in ['granola', 'cereal-bar', 'energy-bar', 'protein-bar', 'snack-bar', 'cereal', 'oat', 'muesli']):
        return 'Bars & Granola'
    if any(k in tags for k in ['nut', 'seed', 'almond', 'peanut', 'cashew', 'pistachio', 'walnut', 'pecan', 'hazelnut']):
        return 'Nuts & Seeds'
    if any(k in tags for k in ['yogurt', 'yoghurt', 'cheese', 'dairy', 'milk', 'cream']):
        return 'Dairy Snacks'
    if any(k in tags for k in ['jerky', 'meat-snack', 'dried-meat', 'meat', 'beef', 'chicken', 'turkey', 'sausage']):
        return 'Meat Snacks'
    if any(k in tags for k in ['fruit', 'dried-fruit', 'raisin', 'berry', 'apple', 'mango', 'banana']):
        return 'Fruit Snacks'
    return 'Other'

df_clean['primary_category'] = df_clean['categories_tags'].apply(assign_category)

print(df_clean['primary_category'].value_counts())
```

```
primary_category
Other                 55091
Bars & Granola         8196
Dairy Snacks           5349
Candy & Chocolate      5216
Fruit Snacks           2815
Meat Snacks            2342
Chips & Crisps         1575
Nuts & Seeds           1373
Cookies & Biscuits      250
Name: count, dtype: int64
```

Products in "Other" category now dropped from 65,916 to 55,091. This means that ~10,000 products were put in proper categories. The remaining products in "Other" category are genuinely unrecognized products with tags like 'en:beverages, en:frozen-foods, en:condiments'

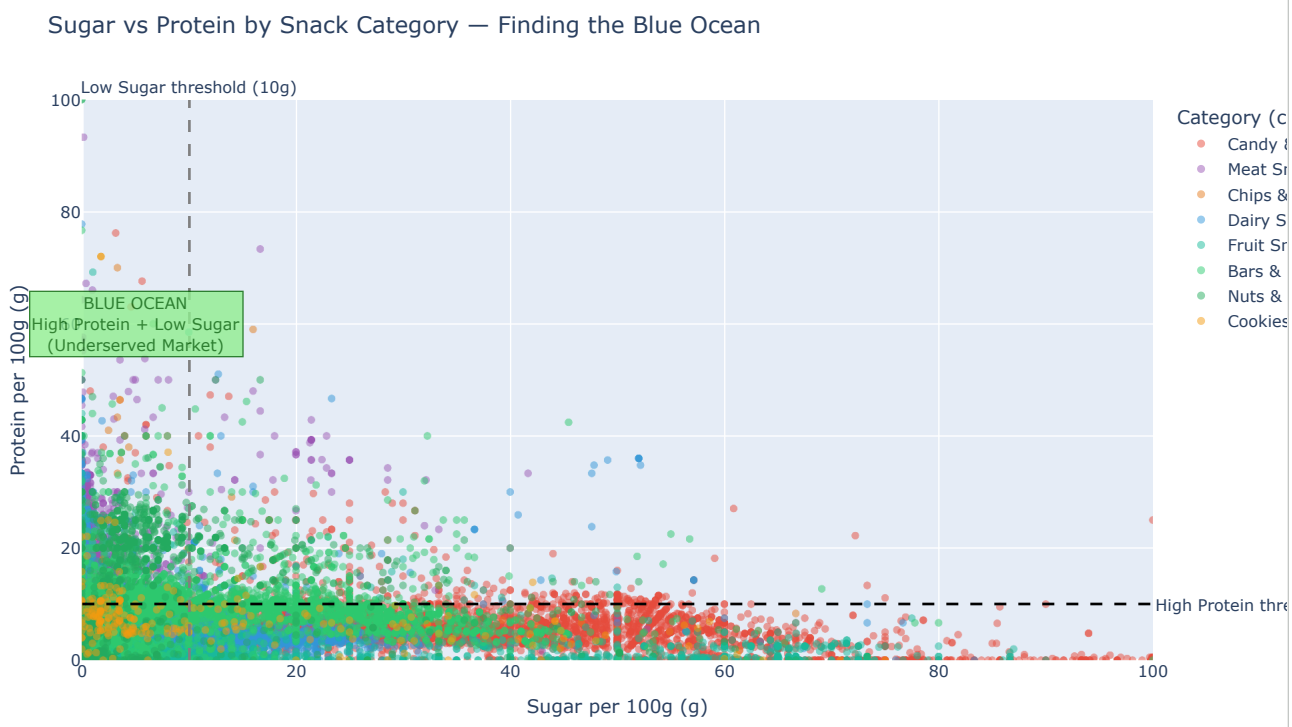
Exclude "Other" from the dashboard

```
df_viz = df_clean[df_clean['primary_category'] != 'Other'].copy()
print(df_viz['primary_category'].value_counts())
print(f"\nTotal products for visualization: {df_viz.shape[0]}")
```

```
primary_category
Bars & Granola        8196
Dairy Snacks          5349
Candy & Chocolate     5216
Fruit Snacks          2815
Meat Snacks           2342
Chips & Crisps        1575
Nuts & Seeds          1373
Cookies & Biscuits     250
Name: count, dtype: int64

Total products for visualization: 27116
```

Story 3: The "Nutrient Matrix" Visualization

```
import plotly.express as px

# Using plotly for intaractive scatter plot
fig = px.scatter(
    df_viz,
    x='sugars_100g',
    y='proteins_100g',
    color='primary_category',
    hover_data=['product_name'],
    opacity=0.5,
    title='Sugar vs Protein by Snack Category — Finding the Blue Ocean',
    labels={
        'sugars_100g': 'Sugar per 100g (g)',
        'proteins_100g': 'Protein per 100g (g)',
        'primary_category': 'Category'
    },
    color_discrete_map={
        'Candy & Chocolate': '#e74c3c',
        'Chips & Crisps': '#e67e22',
        'Cookies & Biscuits': '#f39c12',
        'Bars & Granola': '#2ecc71',
        'Nuts & Seeds': '#27ae60',
        'Dairy Snacks': '#3498db',
        'Meat Snacks': '#9b59b6',
        'Fruit Snacks': '#1abc9c',
    }
)

# Quadrant lines
fig.add_hline(y=10, line_dash="dash", line_color="black",
              annotation_text="High Protein threshold (10g)", annotation_position="right")
fig.add_vline(x=10, line_dash="dash", line_color="gray",
              annotation_text="Low Sugar threshold (10g)", annotation_position="top")

# Blue ocean annotation
fig.add_annotation(
    x=5, y=60,
    text="BLUE OCEAN<br>High Protein + Low Sugar<br>(Underserved Market)",
    showarrow=False,
    font=dict(size=12, color="darkgreen"),
    bgcolor="lightgreen",
    bordercolor="darkgreen",
    borderwidth=1,
    opacity=0.8
)

fig.update_layout(
    xaxis_range=[0, 100],
    yaxis_range=[0, 100],
    legend_title="Category (click to filter)",
    height=600
)

fig.show()
```



Story 4: The Recommendation

```
# Define the blue ocean quadrant
blue_ocean = df_viz[
    (df_viz['proteins_100g'] >= 10) &
    (df_viz['sugars_100g'] <= 10)
]

print("Products in Blue Ocean quadrant by category:")
print(blue_ocean['primary_category'].value_counts())

print("\n── Average nutrition per category in Blue Ocean ──")
print(blue_ocean.groupby('primary_category')[['proteins_100g', 'sugars_100g']].mean().round(1))

print(f"\nTotal blue ocean products: {len(blue_ocean)}")
print(f"That's only {len(blue_ocean)/len(df_viz)*100:.1f}% of all snacks — confirming the gap")
```

```
Products in Blue Ocean quadrant by category:
primary_category
Bars & Granola        2352
Meat Snacks           1780
Dairy Snacks          1243
Nuts & Seeds           761
Chips & Crisps         243
Candy & Chocolate      186
Fruit Snacks            48
Cookies & Biscuits      46
Name: count, dtype: int64

── Average nutrition per category in Blue Ocean ──
                    proteins_100g  sugars_100g
primary_category
Bars & Granola               12.9          3.6
Candy & Chocolate            17.1          2.4
Chips & Crisps               13.2          2.4
Cookies & Biscuits           16.2          2.8
Dairy Snacks                 19.7          1.8
Fruit Snacks                 21.8          2.9
Meat Snacks                  18.5          1.3
Nuts & Seeds                 21.4          4.7

Total blue ocean products: 6659
That's only 24.6% of all snacks — confirming the gap
```

The above cell gives us clear insights on which category dominating the blue ocean and shows us the specific protein/sugar numbers for the recommendation.

Bars & Granola is the biggest opportunity, it has the most blue ocean products (2,352) meaning there's already some consumer demand and product experimentation there, but the overall category is still dominated by high-sugar products. That's where a new entrant can win.

Story 5: Hidden Gem — Top Protein Sources

```
protein_keywords = ['whey', 'peanut', 'soy', 'almond', 'egg',
                    'casein', 'hemp', 'chickpea', 'lentil', 'quinoa',
                    'oat', 'milk', 'rice', 'sunflower', 'pea protein']

# Focusing on blue ocean Bars & Granola products with ingredients
high_protein_bars = blue_ocean[
    (blue_ocean['primary_category'] == 'Bars & Granola') &
    (blue_ocean['ingredients_text'].notna())
]

from collections import Counter
counts = Counter()

for text in high_protein_bars['ingredients_text']:
    text_lower = text.lower()
    for kw in protein_keywords:
        if kw in text_lower:
            counts[kw] += 1

print(f"Analyzed {len(high_protein_bars)} high-protein, low-sugar bars\n")
print("Top protein sources found in ingredients:")
for ingredient, count in counts.most_common(5):
    print(f"  {ingredient}: {count} products")
```

```
Analyzed 1978 high-protein, low-sugar bars

Top protein sources found in ingredients:
  soy: 971 products
  oat: 588 products
  sunflower: 457 products
  milk: 375 products
  rice: 205 products
```

Downloading the clean dataset for streamlit dashboard